



Universidade Federal de Campina Grande (UFCG)

Unidade Acadêmica de Engenharia Elétrica (UAEE)

Sistemas em Tempo Real

Prof. Kyller Costa Gorgonio

Relatório técnico

Estação Inteligente de Carregamento de Veículos Elétricos

Gildemberg Barbosa Santos - 122110462

Luiz Gustavo de Lima Rodrigues - 121110728

Vitória Jamille Gonçalves de Oliveira Nóbrega - 120210601

Campina Grande - PB

27 de julho de 2026

Conteúdo

1	Introdução	3
2	Fundamentação Teórica	4
2.1	Sistemas Operacionais de Tempo Real	4
2.2	FreeRTOS	5
2.3	Tarefas	5
2.4	Filas de Mensagens	6
2.5	Semáforos Contadores	6
2.6	Interrupções de Hardware	7
2.7	GPIO da ESP32	8
3	Descrição do Projeto	8
3.1	Arquitetura Geral	8
3.2	Estrutura do Software	10
3.3	Estrutura de Hardware	11
4	Funcionamento da Estação de Carregamento	11
4.1	Chegada dos Veículos	12
4.2	Classificação das Prioridades	12
4.3	Tempo de Carregamento	13
4.4	Gerenciamento das Filas	14
4.5	Controle dos Carregadores por Semáforo	14
4.6	Sinalização dos Carregadores	15
4.7	Fluxo de Atendimento	15
5	Resultados Experimentais	17
5.1	Validação dos Botões e das Interrupções	17
5.2	Validação das Prioridades	17
5.3	Validação do Tempo de Carregamento	18
5.4	Validação dos LEDs	18
5.5	Validação da Concorrência	18

6	Discussão dos Resultados	19
7	Conclusão	20
	Referências	21

1 Introdução

Os Sistemas Operacionais de Tempo Real (RTOS – *Real-Time Operating Systems*) são empregados em aplicações embarcadas que exigem execução concorrente de múltiplas tarefas, gerenciamento eficiente de recursos compartilhados e atendimento previsível a eventos externos. Diferentemente dos sistemas operacionais de propósito geral, um RTOS prioriza a previsibilidade temporal e o cumprimento dos requisitos de cada tarefa, características importantes em aplicações de controle, automação e sistemas embarcados [1], [2].

Neste contexto, este trabalho apresenta o desenvolvimento de uma estação inteligente de carregamento para veículos elétricos utilizando o FreeRTOS executado em uma placa ESP32 DevKit equipada com o módulo ESP-WROOM-32. O objetivo é simular o funcionamento de uma estação composta por múltiplos carregadores, permitindo analisar aspectos relacionados ao escalonamento de tarefas, à sincronização por semáforos, à comunicação entre tarefas por filas e ao tratamento de diferentes níveis de prioridade.

Inicialmente, a aplicação foi desenvolvida em ambiente Linux utilizando a biblioteca POSIX Threads, possibilitando a validação preliminar da arquitetura de software e dos mecanismos de sincronização. Posteriormente, o sistema foi reestruturado para utilizar os recursos nativos do FreeRTOS e executado em hardware real por meio da plataforma ESP32 e do framework ESP-IDF [3].

Para tornar a aplicação interativa, foram utilizados três botões conectados às entradas digitais da ESP32. Cada botão representa a chegada de um tipo de veículo: normal, com bateria crítica ou de emergência. Dessa forma, os eventos podem ser produzidos de maneira controlada durante os testes, permitindo verificar a política de prioridades implementada.

Três LEDs foram empregados para representar visualmente o estado dos carregadores.

Quando um carregador está disponível, seu LED permanece aceso. Quando um veículo inicia o carregamento, o LED correspondente é apagado, indicando que o recurso está ocupado até a conclusão do atendimento.

O tempo de carregamento não é fixo, sendo calculado dinamicamente em função do nível inicial da bateria. Assim, veículos com menor carga permanecem mais tempo nos carregadores, produzindo diferentes tempos de ocupação dos recursos e influenciando a formação das filas de espera.

O projeto demonstra a aplicação prática dos principais mecanismos oferecidos pelo FreeRTOS, integrando tarefas concorrentes, filas, semáforos contadores, interrupções e periféricos da ESP32 em uma solução embarcada para gerenciamento de uma estação de carregamento.

2 Fundamentação Teórica

2.1 Sistemas Operacionais de Tempo Real

Um Sistema Operacional de Tempo Real é desenvolvido para executar aplicações que possuem requisitos temporais definidos. Nessas aplicações, não basta que o resultado produzido esteja logicamente correto; é também necessário que ele seja obtido dentro de um intervalo de tempo compatível com os requisitos do sistema.

Diferentemente dos sistemas operacionais de propósito geral, cujo objetivo normalmente envolve maximizar o desempenho médio e compartilhar os recursos entre diversos programas, um RTOS prioriza a previsibilidade da execução e a resposta aos eventos importantes da aplicação [1].

Em sistemas embarcados, como controladores industriais, equipamentos médicos, sistemas automotivos e dispositivos de Internet das Coisas, o emprego de um RTOS permite

dividir a aplicação em tarefas independentes, simplificando o desenvolvimento, a manutenção e a expansão do software.

2.2 FreeRTOS

O FreeRTOS é um sistema operacional de tempo real amplamente utilizado em microcontroladores devido ao seu reduzido consumo de memória, portabilidade e disponibilidade de mecanismos de gerenciamento de tarefas, comunicação e sincronização [2], [4].

O escalonador do FreeRTOS seleciona a tarefa que deve utilizar o processador com base em sua prioridade e em seu estado de execução. Uma tarefa pode estar executando, pronta, bloqueada ou suspensa. Quando uma tarefa aguarda um evento, uma mensagem ou um intervalo temporal, ela pode ser colocada no estado bloqueado, permitindo que outras tarefas utilizem o processador.

Neste projeto, o FreeRTOS foi utilizado por meio do ESP-IDF, ambiente oficial de desenvolvimento da Espressif para a família ESP32 [3].

2.3 Tarefas

No FreeRTOS, cada atividade concorrente da aplicação pode ser implementada como uma tarefa. Cada tarefa possui uma função associada, uma região de pilha, uma prioridade e estruturas internas utilizadas pelo escalonador.

Na estação desenvolvida, foram implementadas as seguintes tarefas principais:

- tarefa de chegada dos veículos;
- tarefa de gerenciamento da estação;
- três tarefas independentes de carregamento.

A tarefa de chegada permanece bloqueada aguardando eventos produzidos pelos botões. A tarefa da estação seleciona os veículos conforme a prioridade e aguarda a disponibilidade de um carregador. As três tarefas dos carregadores executam simultaneamente, permitindo até três atendimentos concorrentes.

2.4 Filas de Mensagens

As filas do FreeRTOS permitem a comunicação segura entre tarefas e entre rotinas de interrupção e tarefas. Os elementos enviados são copiados para uma área interna da fila, evitando a necessidade de compartilhamento direto de uma mesma variável entre diferentes contextos de execução [4].

No sistema desenvolvido, as filas são utilizadas para:

- transportar os eventos produzidos pelas interrupções dos botões;
- armazenar separadamente veículos de emergência, críticos e normais;
- encaminhar veículos selecionados pela estação às tarefas dos carregadores.

As filas de prioridade permanecem ocultas dentro do módulo responsável pelo gerenciamento da espera, reduzindo o acoplamento entre os demais arquivos da aplicação.

2.5 Semáforos Contadores

Um semáforo contador representa uma quantidade finita de ocorrências ou recursos. Diferentemente de um semáforo binário, seu valor pode variar entre zero e um limite máximo.

Foram utilizados dois semáforos contadores na aplicação:

- um semáforo que representa a quantidade total de veículos aguardando nas três filas de prioridade;
- um semáforo que representa a quantidade de carregadores disponíveis.

O primeiro permite que a tarefa da estação permaneça bloqueada enquanto não houver veículos aguardando. O segundo é inicializado com valor igual a três, correspondente ao número de carregadores. Sempre que um veículo é encaminhado para atendimento, uma unidade é consumida. Ao término do carregamento, a tarefa responsável devolve essa unidade.

Esse mecanismo garante que o número de veículos carregando simultaneamente nunca ultrapasse a quantidade de carregadores existentes.

2.6 Interrupções de Hardware

As interrupções permitem que o microcontrolador responda à ocorrência de eventos externos sem realizar consultas contínuas ao estado dos periféricos.

Na ESP32, os botões foram configurados para gerar interrupções na transição do nível lógico alto para o nível baixo. Esse comportamento ocorre porque os GPIOs utilizam resistores de *pull-up* internos e os botões são conectados entre as entradas e o terra.

A rotina de interrupção não cria diretamente o veículo nem realiza impressões no terminal. Ela apenas envia um evento para uma fila por meio da função *xQueueSendFromISR()*. A criação do veículo é realizada posteriormente por uma tarefa do FreeRTOS.

Essa organização reduz o tempo de execução da rotina de interrupção e evita a realização de operações complexas em um contexto de alta prioridade.

2.7 GPIO da ESP32

A ESP32 disponibiliza portas de Entrada e Saída de Propósito Geral (GPIO – *General Purpose Input/Output*), que permitem a conexão de sensores, atuadores, botões e LEDs [5], [6].

Neste trabalho foram utilizados três GPIOs de entrada para os botões e três GPIOs de saída para os LEDs. As entradas foram configuradas com *pull-up* interno e interrupção por borda de descida. As saídas digitais foram utilizadas para indicar o estado dos carregadores.

3 Descrição do Projeto

O projeto consiste na implementação de uma estação inteligente de carregamento para veículos elétricos utilizando uma placa ESP32 DevKit com módulo ESP-WROOM-32 e o FreeRTOS disponibilizado pelo ESP-IDF.

A aplicação foi concebida de forma modular, dividindo suas responsabilidades em diferentes arquivos e tarefas. Essa abordagem permite representar situações concorrentes, como a chegada de novos veículos, a formação das filas de espera e o carregamento simultâneo de até três veículos.

Inicialmente, a arquitetura foi validada em ambiente Linux utilizando Pthreads e semáforos POSIX. Após essa etapa, a solução foi reestruturada para utilizar as primitivas próprias do FreeRTOS, especialmente tarefas, filas e semáforos contadores.

3.1 Arquitetura Geral

A arquitetura baseia-se na separação entre os periféricos de hardware, os mecanismos de comunicação do FreeRTOS e as tarefas da aplicação.

Os botões geram interrupções GPIO, que enviam eventos a uma fila. A tarefa de chegada recebe esses eventos, cria os veículos e os insere nas filas de prioridade. A tarefa da estação aguarda um veículo e um carregador disponível, seleciona o veículo de maior prioridade e o envia à fila de atendimento. Por fim, uma das três tarefas de carregamento recebe o veículo e controla o LED correspondente.

A Figura 1 apresenta a arquitetura geral da solução.

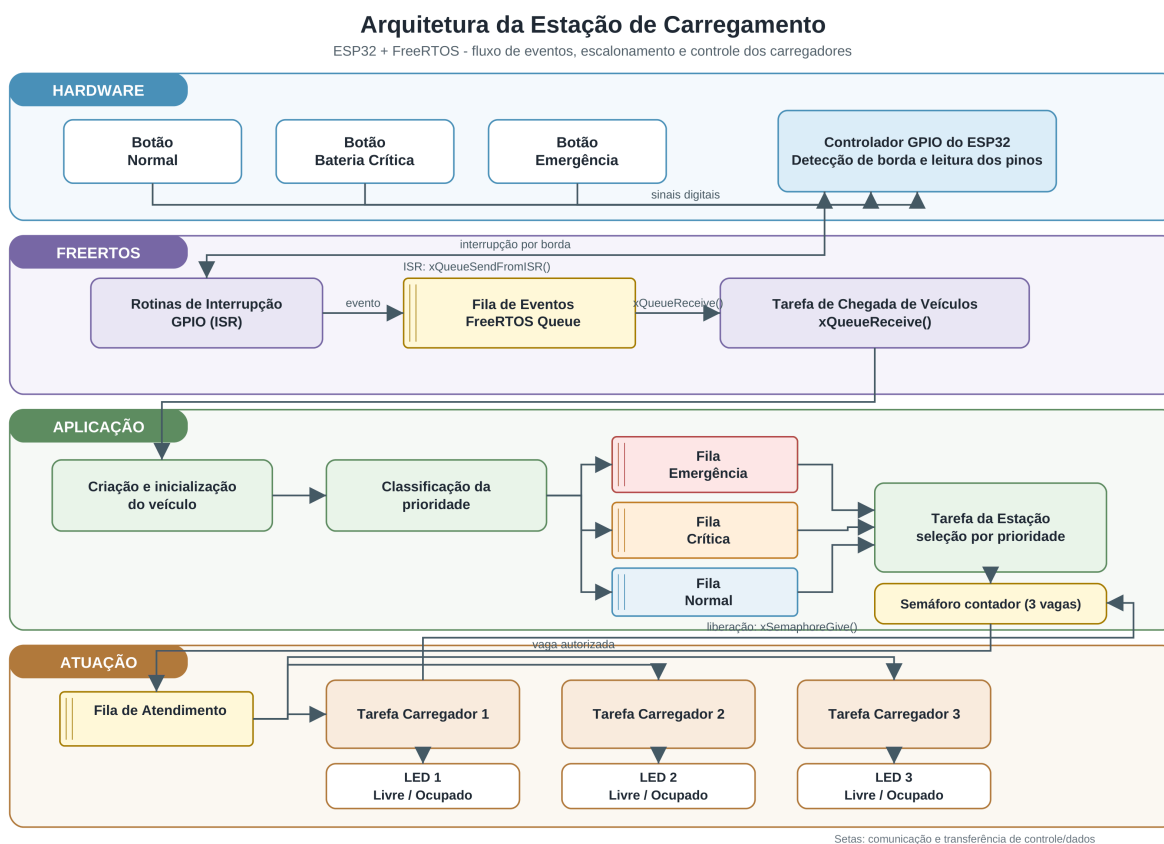


Figura 1: Arquitetura geral da estação de carregamento implementada com ESP32 e FreeRTOS.

3.2 Estrutura do Software

O software foi organizado em módulos independentes. A Tabela 1 apresenta os principais arquivos da aplicação.

Tabela 1: Estrutura modular do software.

Arquivo	Responsabilidade
<code>main.c</code>	Inicialização dos periféricos e da estação.
<code>config.h</code>	Definição de constantes, GPIOs, tamanhos de fila, prioridades e dimensões das pilhas.
<code>veiculo.c</code> / <code>veiculo.h</code>	Definição da estrutura dos veículos, geração do nível de bateria e cálculo do tempo de carregamento.
<code>fila.c</code> / <code>fila.h</code>	Criação das filas de prioridade, inserção dos veículos e seleção do próximo atendimento.
<code>estacao.c</code> / <code>estacao.h</code>	Implementação das tarefas de chegada e gerenciamento da estação.
<code>carregador.c</code> / <code>carregador.h</code>	Criação das tarefas dos carregadores, controle do atendimento e liberação das vagas.
<code>hardware.c</code> / <code>hardware.h</code>	Configuração dos botões, interrupções GPIO, fila de eventos e LEDs.

A separação reduz o acoplamento entre os componentes. Por exemplo, a tarefa da estação não acessa diretamente as estruturas internas das filas, utilizando funções públicas disponibilizadas pelo módulo correspondente.

3.3 Estrutura de Hardware

A implementação física foi realizada com uma placa ESP32 DevKit equipada com o módulo ESP-WROOM-32.

Foram utilizados três botões:

- botão de veículo normal;
- botão de veículo com bateria crítica;
- botão de veículo de emergência.

Cada botão é conectado entre um GPIO e o terra. Como as entradas utilizam o *pull-up* interno, o estado normal do GPIO é alto e o pressionamento produz uma transição para o nível baixo.

Também foram utilizados três LEDs externos, cada um associado a um carregador. Os LEDs foram conectados aos GPIOs de saída por meio de resistores limitadores de corrente.

Quando um carregador está disponível, seu LED permanece aceso. Durante o carregamento, o LED correspondente é apagado, indicando a ocupação do recurso.

4 Funcionamento da Estação de Carregamento

Após a inicialização, a ESP32 configura os botões, os LEDs, as filas e os semáforos. Em seguida, são criadas as tarefas responsáveis pela chegada dos veículos, pelo gerenciamento da estação e pelos três carregadores.

O funcionamento é orientado a eventos. Os veículos somente são criados quando um dos botões é pressionado. Cada veículo percorre as etapas de chegada, criação, classificação, espera, seleção, carregamento e liberação do carregador.

4.1 Chegada dos Veículos

A chegada é simulada por três botões conectados às entradas digitais da ESP32. Cada botão representa uma categoria de veículo e gera uma interrupção quando pressionado.

A rotina de interrupção identifica o botão acionado e envia um elemento para a fila de eventos utilizando *xQueueSendFromISR()*. Quando necessário, a rotina solicita uma troca de contexto para permitir a execução imediata da tarefa que estava bloqueada aguardando o evento.

A tarefa de chegada recebe o evento, define o tipo do veículo, gera um identificador, atribui um nível de bateria e calcula seu tempo de carregamento.

Também foi implementado um tratamento de *debounce* baseado em tempo. Eventos repetidos do mesmo botão dentro de uma janela definida são ignorados, reduzindo a possibilidade de um único pressionamento criar múltiplos veículos.

4.2 Classificação das Prioridades

Foram adotados três níveis de prioridade:

1. emergência;
2. bateria crítica;
3. normal.

Os veículos de emergência possuem prioridade máxima, independentemente do nível de bateria. Os veículos críticos possuem carga inicial entre 10% e 30%. Os veículos normais possuem carga superior ao limite crítico.

Os veículos são armazenados em três filas independentes. A tarefa da estação verifica inicialmente a fila de emergência, seguida pela fila de bateria crítica e, por último, pela fila normal.

A prioridade dos veículos não deve ser confundida com a prioridade das tarefas do FreeRTOS. A primeira determina a ordem de atendimento na estação. A segunda influencia a escolha da tarefa que recebe o processador pelo escalonador.

4.3 Tempo de Carregamento

O tempo de carregamento é calculado de acordo com o nível inicial da bateria. O modelo utilizado é dado por

$$T = (100 - B)K, \tag{1}$$

em que:

- T é o tempo de carregamento, em milissegundos;
- B é o nível inicial da bateria, em porcentagem;
- K é o tempo simulado necessário para carregar um ponto percentual.

Na implementação, foi utilizado

$$K = 100 \text{ ms}/\%. \tag{2}$$

Por exemplo, para um veículo com bateria inicial de 20%:

$$T = (100 - 20) \times 100 = 8000 \text{ ms}. \tag{3}$$

Assim, o tempo simulado de carregamento é igual a 8 segundos.

A relação empregada é uma simplificação linear. Ela não representa todas as características eletroquímicas de uma bateria real, como variação da potência e redução da corrente próxima à carga máxima. Entretanto, é suficiente para introduzir diferentes tempos de ocupação dos carregadores e avaliar a concorrência entre as tarefas.

4.4 Gerenciamento das Filas

Cada veículo é inserido na fila correspondente à sua categoria. Enquanto os três carregadores estão ocupados, os veículos permanecem aguardando.

Um semáforo contador representa a quantidade total de veículos armazenados nas três filas. A tarefa da estação executa uma operação de espera nesse semáforo, permanecendo bloqueada quando não existem veículos disponíveis.

Quando uma vaga é liberada, a tarefa verifica as filas na seguinte ordem:

$$\text{Emergência} > \text{Bateria crítica} > \text{Normal}. \quad (4)$$

Dentro de uma mesma categoria, os veículos são atendidos na ordem de inserção, seguindo o comportamento FIFO (*First In, First Out*) da fila do FreeRTOS.

4.5 Controle dos Carregadores por Semáforo

A estação possui três carregadores, representados por três tarefas independentes. Para controlar esse recurso limitado, foi utilizado um semáforo contador inicializado com valor igual a três.

A tarefa da estação somente remove um veículo das filas depois de obter uma unidade do semáforo de vagas. Essa ordem é importante porque mantém os veículos nas filas de

prioridade enquanto todos os carregadores estão ocupados.

Quando uma vaga é obtida, o veículo de maior prioridade é removido e encaminhado à fila compartilhada de atendimento. Uma das tarefas de carregador recebe esse veículo e inicia o carregamento.

Ao término do tempo calculado, a tarefa libera uma unidade do semáforo. A tarefa da estação pode então encaminhar o próximo veículo prioritário.

4.6 Sinalização dos Carregadores

Os LEDs fornecem uma representação física do estado dos carregadores:

- LED aceso: carregador disponível;
- LED apagado: carregador ocupado.

Durante a inicialização, os três LEDs são acesos. Quando uma tarefa de carregador recebe um veículo, o LED associado é apagado. Após o término do carregamento, o LED é aceso novamente antes da liberação da vaga.

Essa sinalização permite verificar visualmente o funcionamento simultâneo dos três carregadores sem depender exclusivamente das mensagens do monitor serial.

4.7 Fluxo de Atendimento

A Figura 2 apresenta o caminho completo percorrido por um veículo, desde o acionamento do botão até a liberação do carregador.

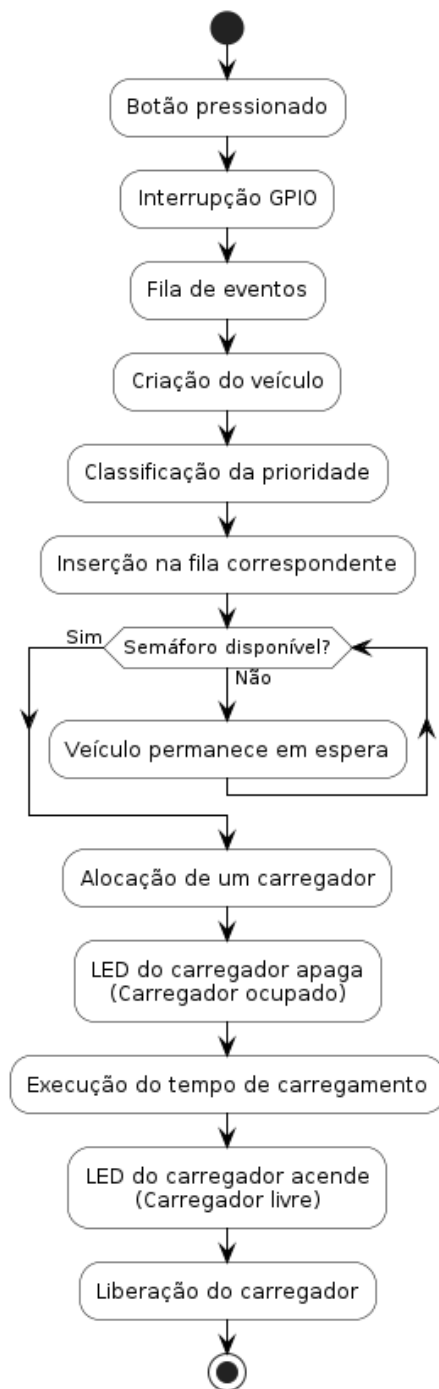
Fluxo de Atendimento de um Veículo

Figura 2: Fluxo completo de atendimento de um veículo na estação.

5 Resultados Experimentais

Após a implementação, foram realizados testes para verificar o funcionamento das tarefas, filas, semáforos, interrupções e periféricos.

5.1 Validação dos Botões e das Interrupções

O sistema respondeu corretamente ao acionamento dos três botões. Cada pressionamento válido produziu um único evento após o tratamento de *debounce* e resultou na criação de um veículo da categoria correspondente.

A rotina de interrupção permaneceu limitada ao envio de eventos para a fila, enquanto a criação do veículo, o cálculo do tempo e as mensagens de acompanhamento foram executados pela tarefa de chegada.

5.2 Validação das Prioridades

Para verificar a política de prioridade, os três carregadores foram inicialmente ocupados com veículos normais. Em seguida, foram inseridos novos veículos normais e um veículo de emergência.

Quando uma vaga foi liberada, o veículo de emergência foi selecionado antes dos veículos normais que já aguardavam. De forma semelhante, veículos com bateria crítica foram selecionados antes dos veículos normais quando não havia emergências na fila.

Os testes confirmaram a seguinte ordem de atendimento:

$$\text{Emergência} > \text{Bateria crítica} > \text{Normal.} \quad (5)$$

5.3 Validação do Tempo de Carregamento

Os resultados apresentados no monitor serial demonstraram que veículos com menor nível de bateria permaneceram mais tempo nos carregadores.

Por exemplo, um veículo com 12% de bateria recebeu um tempo simulado de 8,8 segundos, enquanto um veículo com 64% recebeu 3,6 segundos. Esses resultados estão de acordo com a Equação 1.

Como cada carregador executa em uma tarefa independente, a conclusão dos carregamentos ocorreu em instantes distintos, de acordo com a bateria de cada veículo.

5.4 Validação dos LEDs

Os três LEDs refletiram corretamente o estado das tarefas dos carregadores. Durante períodos sem veículos, os LEDs permaneceram acesos. Ao iniciar um atendimento, o LED correspondente foi apagado e permaneceu nesse estado durante todo o tempo de carregamento.

Após a conclusão, o LED foi aceso novamente, indicando a disponibilidade da vaga.

5.5 Validação da Concorrência

Os testes demonstraram a execução simultânea de até três carregamentos. Enquanto as tarefas de carregador permaneciam bloqueadas por meio de *vTaskDelay()*, a tarefa de chegada continuava respondendo aos botões e a tarefa da estação permanecia apta a organizar as filas.

As filas garantiram a comunicação segura entre os contextos de interrupção e as tarefas. Os semáforos contadores impediram que mais de três veículos fossem encaminhados simultaneamente aos carregadores.

6 Discussão dos Resultados

A arquitetura baseada em tarefas permanentes mostrou-se mais adequada ao ambiente embarcado do que a criação de uma tarefa para cada veículo. Os veículos são representados como estruturas de dados enviadas pelas filas, mantendo fixa a quantidade de tarefas e tornando o consumo de memória mais previsível.

A separação entre a interrupção e o processamento do evento também se mostrou importante. A rotina de interrupção executa rapidamente, enquanto as operações de criação do veículo e inserção na fila são realizadas no contexto de uma tarefa.

A política de prioridade rígida atende ao requisito de oferecer preferência aos veículos de emergência e de bateria crítica. Entretanto, se veículos prioritários forem inseridos continuamente, os veículos normais podem permanecer por longos períodos aguardando atendimento. Esse comportamento, conhecido como inanição ou *starvation*, constitui uma possível limitação da política atual.

Uma evolução possível seria implementar envelhecimento de prioridade, aumentando gradualmente a prioridade dos veículos de acordo com o tempo de espera.

Outra limitação é a capacidade finita das filas. Quando uma fila atinge sua capacidade máxima, novos veículos daquela categoria não podem ser inseridos. Na implementação com botões, a taxa de chegada é controlada pelo usuário, reduzindo a ocorrência desse problema. Ainda assim, uma versão futura poderia sinalizar a condição de fila cheia por meio de um LED específico, display ou interface de monitoramento.

O cálculo linear do tempo de carregamento também representa uma simplificação. Um modelo mais realista poderia considerar a capacidade da bateria, a potência do carregador e diferentes fases da curva de carregamento.

Apesar dessas limitações, os testes confirmaram que a arquitetura atende aos objetivos propostos e demonstra adequadamente o emprego dos principais mecanismos do FreeRTOS.

7 Conclusão

Neste trabalho foi desenvolvido um sistema embarcado para simulação de uma estação inteligente de carregamento de veículos elétricos utilizando uma placa ESP32 DevKit com módulo ESP-WROOM-32 e o sistema operacional de tempo real FreeRTOS.

A migração da aplicação inicialmente desenvolvida em Linux para uma implementação embarcada permitiu validar em hardware os conceitos estudados na disciplina de Sistemas em Tempo Real.

Foram aplicados mecanismos fundamentais do FreeRTOS, incluindo tarefas concorrentes, filas de mensagens, semáforos contadores e comunicação entre interrupções e tarefas.

A integração com os periféricos da ESP32 permitiu que a chegada dos veículos fosse produzida por botões físicos e que a ocupação dos carregadores fosse representada por LEDs.

O tempo de carregamento variável, calculado a partir do nível inicial da bateria, produziu diferentes períodos de ocupação dos carregadores. Dessa forma, foi possível observar o funcionamento concorrente das tarefas e a formação das filas de espera.

Os resultados demonstraram o correto funcionamento da política de prioridades, garantindo preferência aos veículos de emergência, seguida pelos veículos com bateria crítica e pelos veículos normais.

Conclui-se que a aplicação atingiu os objetivos estabelecidos, permitindo consolidar os conceitos de programação concorrente, sincronização, comunicação entre tarefas, temporização e integração de periféricos em uma plataforma embarcada.

Como trabalhos futuros, podem ser considerados o envelhecimento de prioridade, a coleta de estatísticas de atendimento, a utilização de um display, o monitoramento por Wi-Fi e a implementação de uma interface web para acompanhamento remoto da estação.

Referências

- [1] A. S. Tanenbaum e H. Bos, *Modern Operating Systems*, 4^a ed. Pearson, 2015.
- [2] R. Barry, *Mastering the FreeRTOS Real Time Kernel*, 1^a ed. Real Time Engineers Ltd, 2016.
- [3] Espressif Systems, *ESP-IDF Programming Guide*, Acesso em: 27 jul. 2026, Espressif Systems, 2025. URL: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>.
- [4] FreeRTOS, *FreeRTOS Kernel Developer Documentation*, Acesso em: 27 jul. 2026, Amazon Web Services, 2024. URL: https://www.freertos.org/Documentation/RTOS_book.html.
- [5] Espressif Systems, *ESP-IDF GPIO Driver*, Acesso em: 27 jul. 2026, Espressif Systems, 2025. URL: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/gpio.html>.
- [6] Espressif Systems, *ESP32 Series Datasheet*, Acesso em: 27 jul. 2026, Espressif Systems, 2024. URL: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.